Mastering GitHub Actions Caching For Faster Builds

Waiting for your GitHub Actions to build can waste hours of your work week. Caching saves your project dependencies so your pipelines run lightning fast for free.

What you'll learn:

1. Understanding How Caching Works
2. Caching Node.js Dependencies Easily
3. Using Cache Keys Smartly
4. Checking If Your Cache Worked

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Understanding How Caching Works

Imagine going to the grocery store and buying the same exact items every single day instead of keeping them in your fridge. GitHub Actions acts like an empty fridge on every run by default. Caching lets you store your downloaded packages in a temporary fridge so you don't have to download them again tomorrow.

```
- uses: actions/cache@v3
  with:
    path: ~/.npm
    key: ${{ runner.os }}-node-${{ hashFiles('**/package-lock.json') }}
```

GitHub Actions Caching: Speed Up Your Workflows for Free

Wednesday, October 14, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Caching Node.js Dependencies Easily

Node projects download thousands of small files into node_modules. Instead of running npm install from scratch every time, the official setup-node action has caching built right in. You just add one extra line of configuration to your workflow file.

```
- name: Set up Node.js
  uses: actions/setup-node@v3
  with:
    node-version: '18'
    cache: 'npm'
```

GitHub Actions Caching: Speed Up Your Workflows for Free

Wednesday, October 14, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Using Cache Keys Smartly

GitHub uses a key to find your saved cache later. If your package-lock.json file changes, you want a new cache. We use the hashFiles function to create a unique fingerprint of your dependency file so the cache updates automatically when you add new packages.

```
key: npm-cache-${{ hashFiles('package-lock.json') }}
restore-keys: |
  npm-cache-
```

GitHub Actions Caching: Speed Up Your Workflows for Free

Wednesday, October 14, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Checking If Your Cache Worked

After pushing your code to GitHub, open the Actions tab and click on your running workflow. Look at the step where the cache is loaded. If it says 'Cache restored successfully', you just saved yourself several minutes of waiting time.

```
$Post job cleanup:
 Cache saved successfully with key: Linux-node-abc12345
```

GitHub Actions Caching: Speed Up Your Workflows for Free

Wednesday, October 14, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Always tie your cache key to your dependency lock file.
+ Keep cache paths specific to avoid bloating storage limits.
+ Use built-in caching provided by actions like setup-node or setup-python.
+ Test your workflow twice to make sure the second run actually hits the cache.
+ Keep total cache sizes under the 10GB repository limit.

Watch Out For

! Caching the entire project output folder instead of just dependency folders.
! Forgetting to update the cache key when changing package managers.
! Trying to cache files that change on every single commit.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh